Why these rules

The motivation behind the framework for AI-assisted development

---

A companion text to “Rules for AI-assisted development”

A short reflection on what the framework grew out of - principles, not instructions.

1. Speed is not the same as mastery

AI tools for development assistance dramatically increase the speed at which code comes into being. That is their main benefit - and at the same time their main risk. Speed without understanding does not lead to better software; it leads to software nobody can safely maintain or defend.

The core of the problem is inconspicuous: AI generates code that is syntactically correct and passes basic tests - but on its own it reflects neither architectural decisions, nor the threat model, nor the operational context, unless someone deliberately supplied and verified them. “It works” and “we understand it” are not the same thing. And with AI the gap between them does not widen slowly - it opens at the speed at which code accumulates.

2. What is lost when the mental model disappears

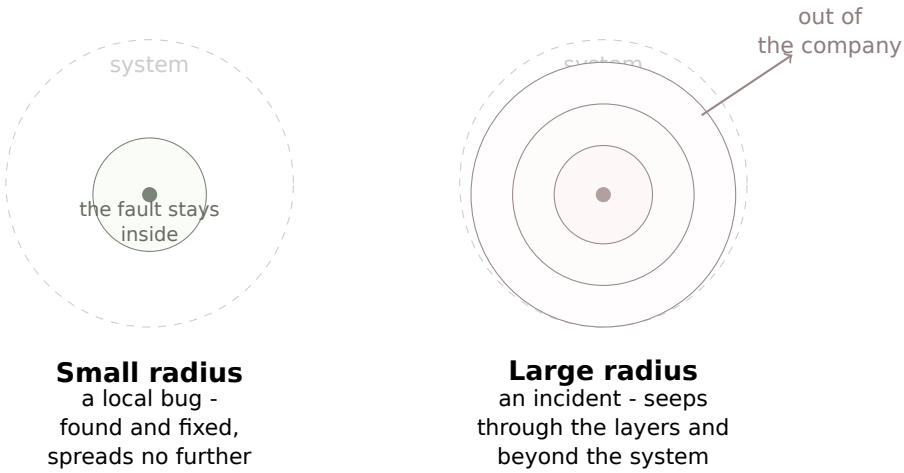
The barrier to entry has fallen so far that a working application can be assembled without anyone going through a phase of deep understanding. The code exists, it runs - but the mental model someone could carry in their head never came into being.

The consequences do not show up straight away. They come later: every non-trivial change becomes a gamble, because it is not clear what affects what. Debugging turns from analysis into trial and error. And when the original author leaves, there is nobody to take the code over - not because it is complicated, but because there never was an understanding that could be handed on.

This is not an AI problem. It is an old problem that AI merely makes faster and cheaper, to the point where it appears more often and sooner.

3. Not every fault is the same

The intuition that “a bug is a bug” is useless in production. A fault in date formatting is local - it shows up, it is found, it is fixed. A fault in how data or access is handled may not show up at all - until it shows up as an incident, months after deployment.



Two faults can be equally “large” in the code and still have completely different reach: one stays where it arose, the other spreads through the layers of the system - and in the worst case all the way out, to users or into data, where it is no longer a bug but an incident. The size of the impact is, however, only the first of three questions.

That is why it makes sense to ask not “who or what wrote the code”, but: how much damage the fault will cause, how quickly we will detect it and how easily we can undo it.

- **Verifiability - how quickly we detect it.** One fault fails a test straight away, another waits quietly. Poor verifiability is insidious precisely because a large radius and late detection multiply each other: by the time the fault surfaces, it is everywhere.

- **Reversibility - how easily we undo it.** A reversible migration or a feature toggle lowers the risk even with a large radius - we simply take the mistake back. An irreversible operation (deleted data without a backup) is dangerous even with a small radius, because there is no second attempt.

These three questions - blast radius, verifiability, reversibility - held long before AI. AI does not change them. It merely raises the stakes by making it possible to produce risk-bearing code faster than ever before.

4. When more than code is at stake

Some areas carry more than the risk of a fault. Where personal data or know-how is involved, carelessness has a legal and commercial dimension too - regulations such as NIS2 or GDPR are not just paperwork, but a description of what happens when the blast radius crosses the boundary of the company. And a tool that sends code or data to a third party for processing is always an outbound channel, not merely a helper.

The details of these topics belong in the rules themselves, not here. It is enough to say here why they matter: because this is where it stops being about the elegance of the code and starts being about responsibility.

5. What the framework is aiming at

The goal is neither to ban AI nor to restrict it. Nor is it a celebration of the tool. It is an attempt to hold a simple balance: generation speed matched by a corresponding degree of understanding, verification and control - most of all where failure hurts most.

An experienced person with AI does more work in less time, but still understands every line they let into production. That is the bar the rules aim at. Everything else is merely a way of holding it.

6. A guide to what already exists and what it aims at

This framework did not arise in a vacuum and does not try to replace work that others have done better. The overview is deliberately here, in the motivational text, and not in the rules: the rules should not change every time a new document comes out. It reflects the state at the time of writing and will go out of date - the rules will not.

Where a document is described from the project's own characterisation rather than from a direct reading, this is noted at the entry.

- **OWASP GenAI Security Project** - a family of documents, not a single text. Each document in it aims somewhere different, which is why it is worth distinguishing them:
  - **Top 10 for LLM Applications** - the oldest and most cited text of the family: the ten most serious risks of language models in applications. The other documents refer to it as their foundation.
  - **Agentic AI - Threats and Mitigations** - a detailed taxonomy of threats to agentic systems (T1–T17). The project's reference vocabulary, a classification rather than a guide.
  - **Top 10 for Agentic Applications** (2025) - ten risks of agentic applications (ASI01–ASI10) in the familiar format. It is meant to serve as a compass; in the introduction the authors themselves admit that the breadth of the project's publications translates poorly into practice. The items closest to this framework are the ones on unexpected code execution and on abuse of trust between human and agent - but both aim at running the agent, not at the moment a change is merged.
  - **Securing Agentic Applications** and **Agentic Threat Modelling Guide** - guides for architecture, design, development, deployment and threat modelling. They enter at the beginning, when the system is being designed.

  - **State of Agentic AI Security and Governance** - an overview of the state of agentic AI governance: what organisations are introducing and where the gaps are. It is precisely here that the need to calibrate the degree of oversight to the consequence is named, without a procedure for doing so being given.
  - **AIVSS** - a scoring system for agentic vulnerabilities. It serves to prioritise problems already found, not to decide where to work carefully in advance.

- **TrustX ARC** (Responsible AI Institute, 2026). A scoring rubric that assigns an agentic system a risk level across twelve dimensions and a set of organisational controls to go with it; it also has a separate extension for coding assistants. The unit of assessment is the *tool*; it enters when the tool is acquired and at periodic reassessment. It presupposes a role that carries out the assessment - at the highest level, up to sign-off by leadership.
- **Security-Focused Guide for AI Code Assistant Instructions** (OpenSSF Best Practices Working Group). A guide to writing instructions for coding assistants, with an extensive set of sample directives from input validation to image signing. It aims directly at developers and, of the whole list, is the closest to what this framework does. The authors themselves point out that the directives are meant to be selected from, not copied wholesale - but they give no criterion for the selection.
- **AI Codebase Maturity Model** (2026). It describes five (six in the newer version) maturity levels of the *feedback infrastructure* around AI - from none, through instruction files and measurement, to closed loops. The unit is the repository as a whole, and the levels are not skipped. Useful for deciding how fast a team can go overall; it does not say where inside the repository things are different.
- **Governed AI-Assisted Engineering (GAIE)** (Kang, arXiv:2606.22484, 2026). Three levels of oversight for agentic code generation in regulated industries - from human in the loop, through human over the loop, to automation with monitoring. The unit is the *task*; classification is decided by regulatory impact, proximity to the customer, reversibility and data sensitivity, and each level prescribes the evidence required for audit. The anchoring is deliberately regulatory: the framework is mapped onto the policy of the Thai central bank and further onto MAS, NIST AI RMF, ISO/IEC 42001 and the AI Act. It therefore presupposes a regulator and a control function to demonstrate this to - where these are absent, what remains of the framework is a classification with no addressee.

General AI risk management frameworks (NIST AI RMF, ISO/IEC 42001, the AI Act) remain outside the scope of this overview. They were not assessed here, and leaving them out is not a judgement - they aim at a different layer and a different reader.

What these documents have in common is that they enter somewhere other than at the merge request: when a tool is acquired, when the architecture is designed, at an audit, while the code is being written. None of them answers the question the developer faces at the moment of merging: *what exactly should I verify in this change, and against what*. That is precisely where this framework stands - and that is why it can be read in two ways:

- **As the only document.** In a small company, or in a department that is effectively a small company inside a large one, none of the roles above exists and nobody will fill them. This framework is then everything that will be introduced - and it is written that way: no new role, no new tool, decidable by whoever does the work.
- **As the missing floor.** Where some governance already exists, this framework replaces none of it. It fills in the one place none of those documents reaches, because none of them aims there - the decision about a specific change, on a specific path in the repository.

What this framework is not meant to be is a mapping table between the others. Claiming that a given rule “corresponds” to a particular provision of someone else’s document is a commitment that goes out of date with its next version and that nobody here would carry. The overview above says what each project aims at - not that they are equivalent to one another.